

run_d2q.py (Q&A Documentation)

Q: What does this run script control? **A:** It controls the full experiment loop: args, data loading, model instantiation, training with a specific loss/optimizer, and evaluation with shared metrics.

Q: Why is the model not responsible for training/evaluation? **A:** Keeping training logic in the script makes it easy to compare models under the same protocol and avoids mixing I/O/metrics with differentiable code.

Q: What does this file export (public API)? **A:** The public API is the set of classes/functions other modules import (sometimes re-exported by package initializers). The key top-level definitions detected here are:

```
def get_args(...) # function
def get_loaders(...) # function
def label_norm(...) # function
def from_value_to_quantile(...) # function
def from_quantile_to_value(...) # function
def get_buckets_infor(...) # function
def mae_rescale_to_second(...) # function
def test(...) # function
```

Q: What are the main dependencies (imports) and why do they matter? **A:** Imports show whether this file is model code (torch), data code (pandas), or evaluation/analysis (sklearn). They also determine runtime requirements.

```
import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataLoader
from model import D2Q, WideAndDeep
from utils import eval_mae, eval_xauc, eval_kl
import pandas as pd
import pickle as pkl
from torch.distributions import Normal
```

```
def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
```

```

parser.add_argument('--device', default='cuda:0')
parser.add_argument('--quantile_max', type=float, default=100)
parser.add_argument('--bsz', type=int, default=2048)
parser.add_argument('--log_interval', type=int, default=10)

parser.add_argument('--epoch', type=int, default=10)
parser.add_argument('--lr', type=float, default=0.1)
parser.add_argument('--weight_decay', type=float, default=1e-6)
parser.add_argument('--seed', type=int, default=42)

args = parser.parse_args()
return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataloader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders

def label_norm(label, max_value):
    return torch.clamp(label, max=max_value)/max_value

def from_value_to_quantile(bucket_quantiles, bucket_index, value):
    quantile = bucket_quantiles[bucket_index]
    if len(quantile) < 2:
        return 0.0
    if value <= quantile[0]:
        return 0.0
    if value >= quantile[-1]:
        return len(quantile) - 1
    idx = np.searchsorted(quantile, value) - 1

```

Q: What is get_args and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for get_args? **A:** Excerpt from the file:

```

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--quantile_max', type=float, default=100)
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

```

Q: What is `get_loaders` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_loaders`? **A:** Excerpt from the file:

```
def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuairc':
        dataloaders = KUAIRECDataloader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unkown dataset name: {}'.format(name))
    return dataloaders
```

Q: What is `label_norm` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `label_norm`? **A:** Excerpt from the file:

```
def label_norm(label, max_value):
    return torch.clamp(label, max=max_value)/max_value
```

Q: What is `from_value_to_quantile` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `from_value_to_quantile`? **A:** Excerpt from the file:

```
def from_value_to_quantile(bucket_quantiles, bucket_index, value):
    quantile = bucket_quantiles[bucket_index]
    if len(quantile) < 2:
        return 0.0
    if value <= quantile[0]:
        return 0.0
    if value >= quantile[-1]:
        return len(quantile) - 1
    idx = np.searchsorted(quantile, value) - 1
    left_val = quantile[idx]
    right_val = quantile[idx + 1]
    if right_val == left_val:
        return float(idx)
    fraction = (value - left_val) / (right_val - left_val)
    quantile = idx + fraction
    return quantile
```

Q: What is `from_quantile_to_value` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `from_quantile_to_value`? **A:** Excerpt from the file:

```
def from_quantile_to_value(bucket_quantiles, bucket_index, quantile):
    quantiles = bucket_quantiles[bucket_index]
    num_quantiles = len(quantiles)
    quantile_steps = np.linspace(0.0, 1.0, num_quantiles)
    if quantile <= 0.0:
        return quantiles[0]
    elif quantile >= 1.0:
        return quantiles[-1]
    idx = np.searchsorted(quantile_steps, quantile) - 1
    if quantile_steps[idx+1] == quantile:
        return quantiles[idx+1]
    q1, q2 = quantile_steps[idx], quantile_steps[idx + 1]
    v1, v2 = quantiles[idx], quantiles[idx + 1]
    interpolated_value = v1 + (v2 - v1) * (quantile - q1) / (q2 - q1)
    return interpolated_value
```

Q: What is `get_buckets_infor` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `get_buckets_infor`? **A:** Excerpt from the file:

```
def get_buckets_infor(buckets_quantiles_path):
    df = pd.read_csv(buckets_quantiles_path)
    bucket_quantiles = {}

    for idx, row in df.iterrows():
        bucket_index = int(row[0])
        quantiles = row[1:].values.tolist()
        bucket_quantiles[bucket_index] = quantiles

    return bucket_quantiles
```

Q: What is `mae_rescale_to_second` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `mae_rescale_to_second`? **A:** Excerpt from the file:

```
def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuaiREC':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae * (6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))
```

Q: What is `test` and what responsibility does it have? **A:** This function is a core unit in the pipeline. Its responsibility is inferred from its name and how run scripts call it.

Q: What does the implementation look like for `test`? **A:** Excerpt from the file:

```
def test(args, model, dataloaders):
    model.eval()
    labels, scores, predicts, durs = list(), list(), list(), list()
```

```

with torch.no_grad():
    for _, (features, label) in enumerate(dataloaders['test']):
        bucket_index = features['duration_bucket']
        y = model(features)
        mapped_y, mapped_label = [], []
        for idx, quantile in zip(bucket_index, y.tolist()):
            mapped_y_value = from_quantile_to_value(buckets_quantiles, idx.item(), quantile)
            mapped_y.append(mapped_y_value)
        labels.extend(label.tolist())
        scores.extend(mapped_y)
        durs.extend(features['duration'].squeeze().tolist())
labels, scores = np.array(labels), np.array(scores)
mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
mae = mae_rescale_to_second(args.dataset_name, mae)
print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))

```

Q: Example: end-to-end data flow involving this file **A: Core pipeline shape (schematic):**

```

python run_d2q.py --dataset_name kuaiREC --dataset_path ./dataset --device cuda:0
# Internally: load dataloaders → build model → train → evaluate

```

Q: Full source code of the Python file **A: The complete file is included verbatim for reference.**

```

import os
import copy
import torch
import random
import numpy as np
import argparse
from dataloader import KUAIRECDataloader
from model import D2Q, WideAndDeep
from utils import eval_mae, eval_xauc, eval_kl
import pandas as pd
import pickle as pkl
from torch.distributions import Normal

def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--dataset_name', default='kuaiREC')
    parser.add_argument('--dataset_path', default='./dataset/')
    parser.add_argument('--device', default='cuda:0')
    parser.add_argument('--quantile_max', type=float, default=100)
    parser.add_argument('--bsz', type=int, default=2048)
    parser.add_argument('--log_interval', type=int, default=10)

    parser.add_argument('--epoch', type=int, default=10)
    parser.add_argument('--lr', type=float, default=0.1)
    parser.add_argument('--weight_decay', type=float, default=1e-6)
    parser.add_argument('--seed', type=int, default=42)

    args = parser.parse_args()
    return args

def get_loaders(name, dataset_path, device, bsz):
    path = os.path.join(dataset_path, name, "{}_data.pkl".format(name))
    if name == 'kuaiREC':
        dataloaders = KUAIRECDataloader(name, path, device, bsz=bsz)
    else:
        raise ValueError('unknown dataset name: {}'.format(name))
    return dataloaders

def label_norm(label, max_value):
    return torch.clamp(label, max=max_value)/max_value

def from_value_to_quantile(bucket_quantiles, bucket_index, value):

```

```

quantile = bucket_quantiles[bucket_index]
if len(quantile) < 2:
    return 0.0
if value <= quantile[0]:
    return 0.0
if value >= quantile[-1]:
    return len(quantile) - 1
idx = np.searchsorted(quantile, value) - 1
left_val = quantile[idx]
right_val = quantile[idx + 1]
if right_val == left_val:
    return float(idx)
fraction = (value - left_val) / (right_val - left_val)
quantile = idx + fraction
return quantile

def from_quantile_to_value(bucket_quantiles, bucket_index, quantile):
    quantiles = bucket_quantiles[bucket_index]
    num_quantiles = len(quantiles)
    quantile_steps = np.linspace(0.0, 1.0, num_quantiles)
    if quantile <= 0.0:
        return quantiles[0]
    elif quantile >= 1.0:
        return quantiles[-1]
    idx = np.searchsorted(quantile_steps, quantile) - 1
    if quantile_steps[idx+1] == quantile:
        return quantiles[idx+1]
    q1, q2 = quantile_steps[idx], quantile_steps[idx + 1]
    v1, v2 = quantiles[idx], quantiles[idx + 1]
    interpolated_value = v1 + (v2 - v1) * (quantile - q1) / (q2 - q1)
    return interpolated_value

def get_buckets_infor(buckets_quantiles_path):
    df = pd.read_csv(buckets_quantiles_path)
    bucket_quantiles = {}

    for idx, row in df.iterrows():
        bucket_index = int(row[0])
        quantiles = row[1:].values.tolist()
        bucket_quantiles[bucket_index] = quantiles

    return bucket_quantiles

def mae_rescale_to_second(dataset, mae):
    if dataset == 'kuaiREC':
        return mae * 999639 / 1000
    elif dataset == 'wechat':
        return mae * 20840
    elif dataset == 'cikm16':
        return (mae * (6000-31) + 31) / 1000
    else:
        raise ValueError('unkown dataset name: {}'.format(dataset))

def test(args, model, dataloaders):
    model.eval()
    labels, scores, predicts, durs = list(), list(), list(), list()
    with torch.no_grad():
        for _, (features, label) in enumerate(dataloaders['test']):
            bucket_index = features['duration_bucket']
            y = model(features)
            mapped_y, mapped_label = [], []
            for idx, quantile in zip(bucket_index, y.tolist()):
                mapped_y_value = from_quantile_to_value(buckets_quantiles, idx.item(), quantile)
                mapped_y.append(mapped_y_value)
            labels.extend(label.tolist())
            scores.extend(mapped_y)
            durs.extend(features['duration'].squeeze().tolist())
    labels, scores = np.array(labels), np.array(scores)

```

```

mae, xauc, kl = eval_mae(labels, scores), eval_xauc(labels, scores), eval_kl(labels, scores)
mae = mae_rescale_to_second(args.dataset_name, mae)
print("test result | MAE: {:.7f} | XAUC: {:.7f} | KL: {:.7f}".format(mae, xauc, kl))

if __name__ == '__main__':
    args = get_args()
    if args.seed > -1:
        np.random.seed(args.seed)
        torch.manual_seed(args.seed)
        torch.cuda.manual_seed(args.seed)
    res = {}
    torch.cuda.empty_cache()

    device = torch.device(args.device)

    dataloaders = get_loaders(args.dataset_name, args.dataset_path, device, args.bsz)
    buckets_quantiles_path = os.path.join(args.dataset_path, args.dataset_name, "d2q_duration_bucket_playtime_quantiles.csv")
    buckets_quantiles = get_buckets_infor(buckets_quantiles_path)
    model = D2Q(dataloaders.description, embed_dim=16, mlp_dims=(512, 256, 128, 64), dropout=0.0)
    model = model.to(device)

    # train
    dataloader_train = dataloaders['train']
    model.train()
    criterion = torch.nn.MSELoss()
    # criterion = torch.nn.BCELoss()
    optimizer = torch.optim.Adagrad(params=model.parameters(), lr=args.lr, weight_decay=args.weight_decay)
    for epoch_i in range(1, args.epoch + 1):
        model.train()
        epoch_loss = 0.0
        total_loss = 0
        total_iters = len(dataloader_train)
        for i, (features, label) in enumerate(dataloader_train):
            y = model(features)
            bucket_index = features['duration_bucket']
            mapped_label = []
            for idx, label_origin in zip(bucket_index, label.tolist()):
                # print("{}, {}".format(idx.item(), label_origin))
                mapped_label_value = from_value_to_quantile(buckets_quantiles, idx.item(), label_origin)
                mapped_label.append(mapped_label_value)
            # print("train orig mapped_label:", mapped_label)
            mapped_label = torch.tensor(mapped_label, device=device)
            mapped_label = label_norm(mapped_label, args.quantile_max)
            # print("y:", y)
            loss = criterion(y, mapped_label.float())
            model.zero_grad()
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()
            total_loss += loss.item()
            if (i + 1) % 10 == 0:
                print("    Iter {}/{} loss: {:.7f}".format(i + 1, total_iters + 1, total_loss/args.log_interval), end='\r')
                total_loss = 0
        print("Epoch {}/{} average Loss: {:.7f}".format(epoch_i, args.epoch, epoch_loss/total_iters))
    test(args, model, dataloaders)

```